Введение в HDFS

Лекция 4

Проблема хранения больших данных

Традиционные система хранения:

1. Локальная файловая система
2. Сетевые хранилища
3. Базы данных

Проблемы:

1. Ограниченный объем
2. Высокая стоимость масштабирования
3. Единая точка отказа системы

BIG DATA vs SMALL DATA

BIG DATA		SMALL DATA
Can be structured, semi-, or un-structured	Structure	Highly structured
Web data, IoT data, etc.	Sources	Sales data, demographic information, etc.
Multiple servers on the cloud	Location	It can be stored on a single computer
Rarely reproducible	Reproducibility	Often reproducible
Usually prepared by multiple users	Data Preparation	Usually prepared by a single user
NoSQL	Database	SQL
Multiple units of measurement	Measurements	A single unit of measurement

Требования к системам хранения Big Data

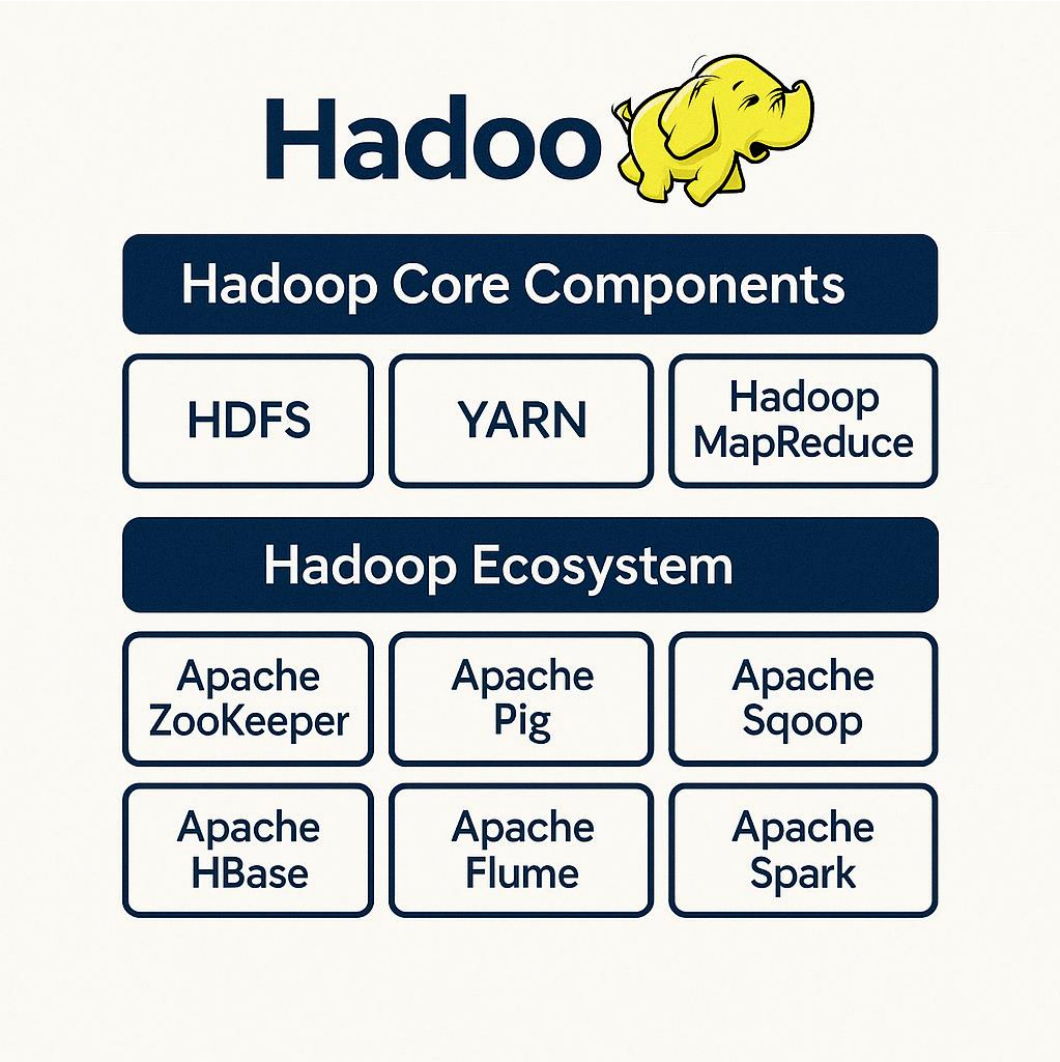
Должны мочь в:

1. Горизонтальное масштабирование
2. Отказоустойчивость
3. Дешевое хранение
4. Параллельный доступ
5. Интеграцию с вычислительными системами

Hadoop Ecosystem

Основные компоненты:

1. HDFS – распределенная файловая система
2. YARN – менеджер ресурсов
3. MapReduce – вычислительный фреймворк



HDFS

HDFS (*Hadoop Distributed File System*) – это **распределённая** файловая система, предназначенная для хранения **больших файлов** на **кластере обычных серверов**.

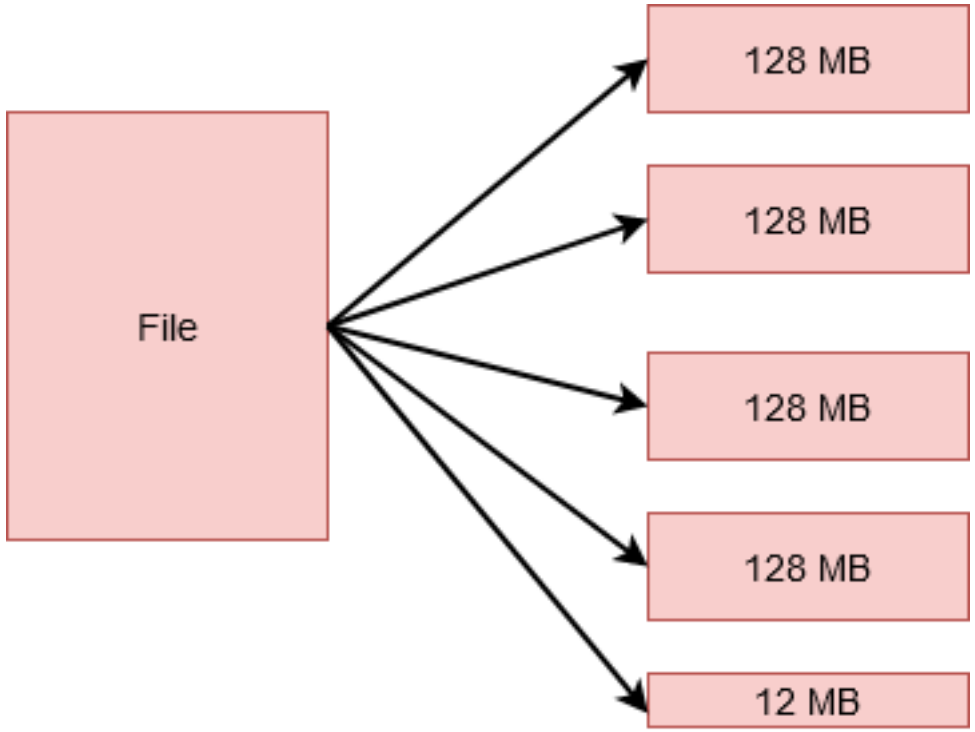
Свойства:

- Масштабируемая
- Отказоустойчивая
- Оптимизированная для больших данных

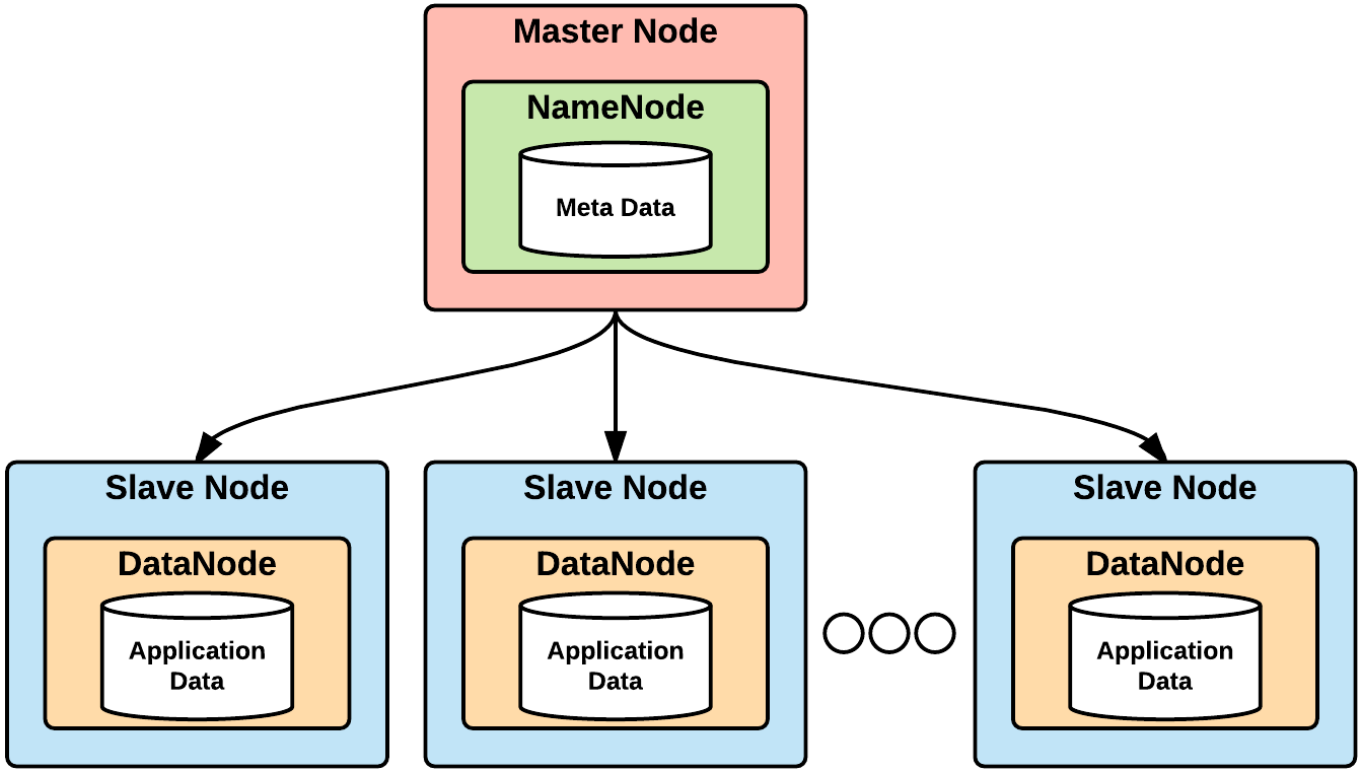
Блоки данных

HDFS режет данные на блоки

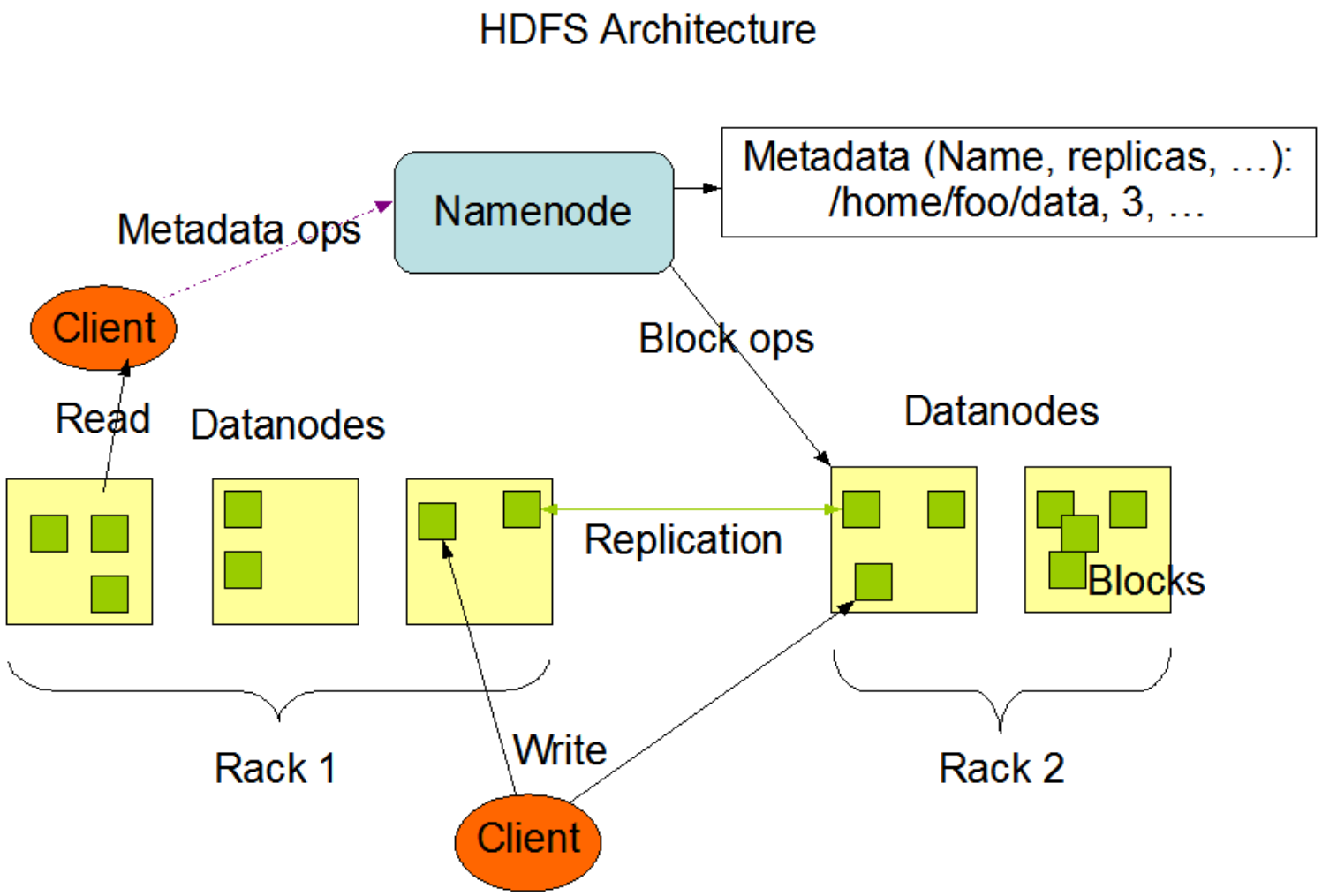
Размер блока по умолчанию – **128 МБ** (можно менять, обычно не меняют)



HDFS архитектурно



HDFS архитектурно подробнее



Архитектура словами

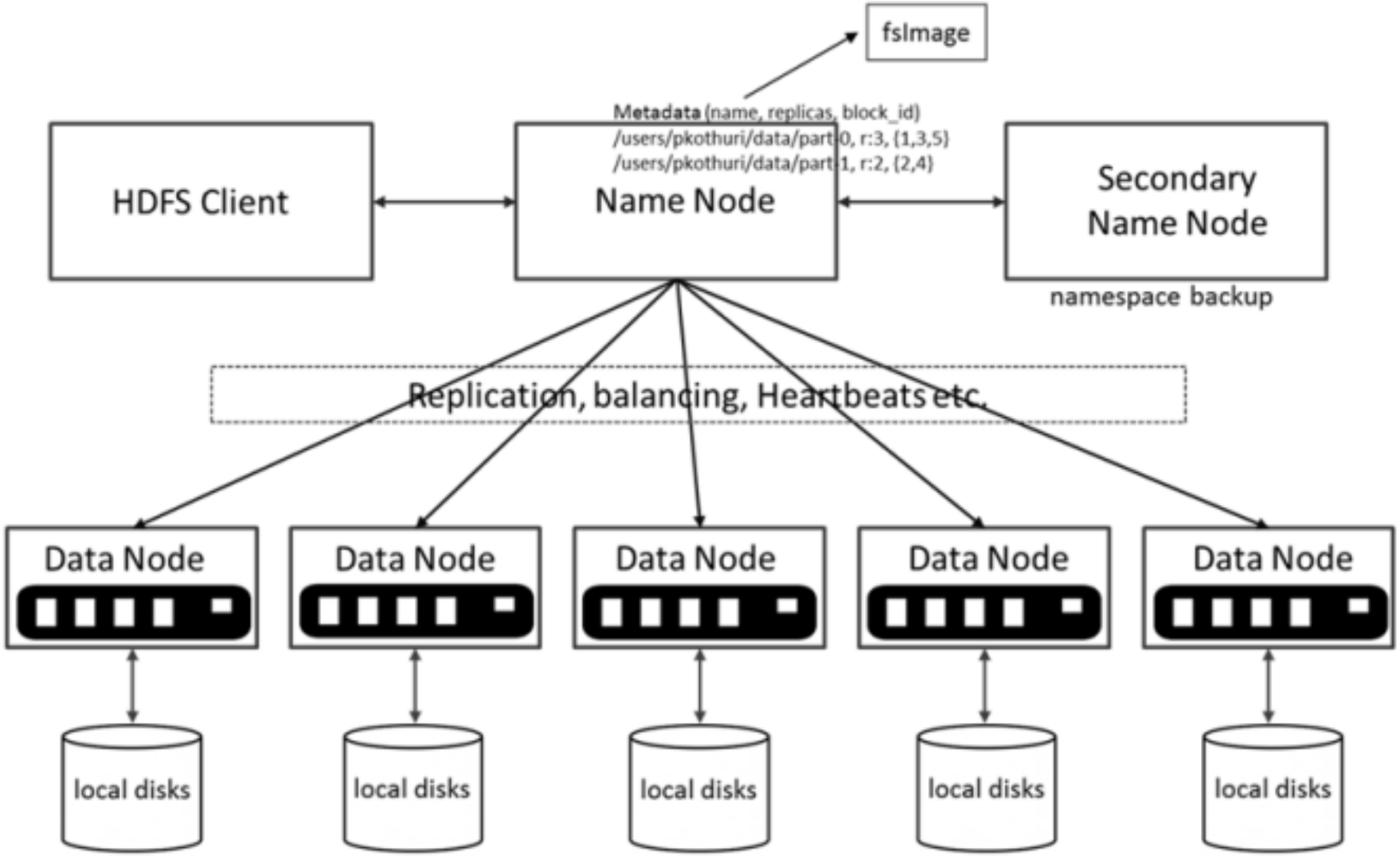
Namenode

- Одна (за исключением второй)
- НЕ хранит данные
- Хранит структуру каталогов, метаданные файлов, расположение блоков, информацию о датанодах
- Управляет файловой системой

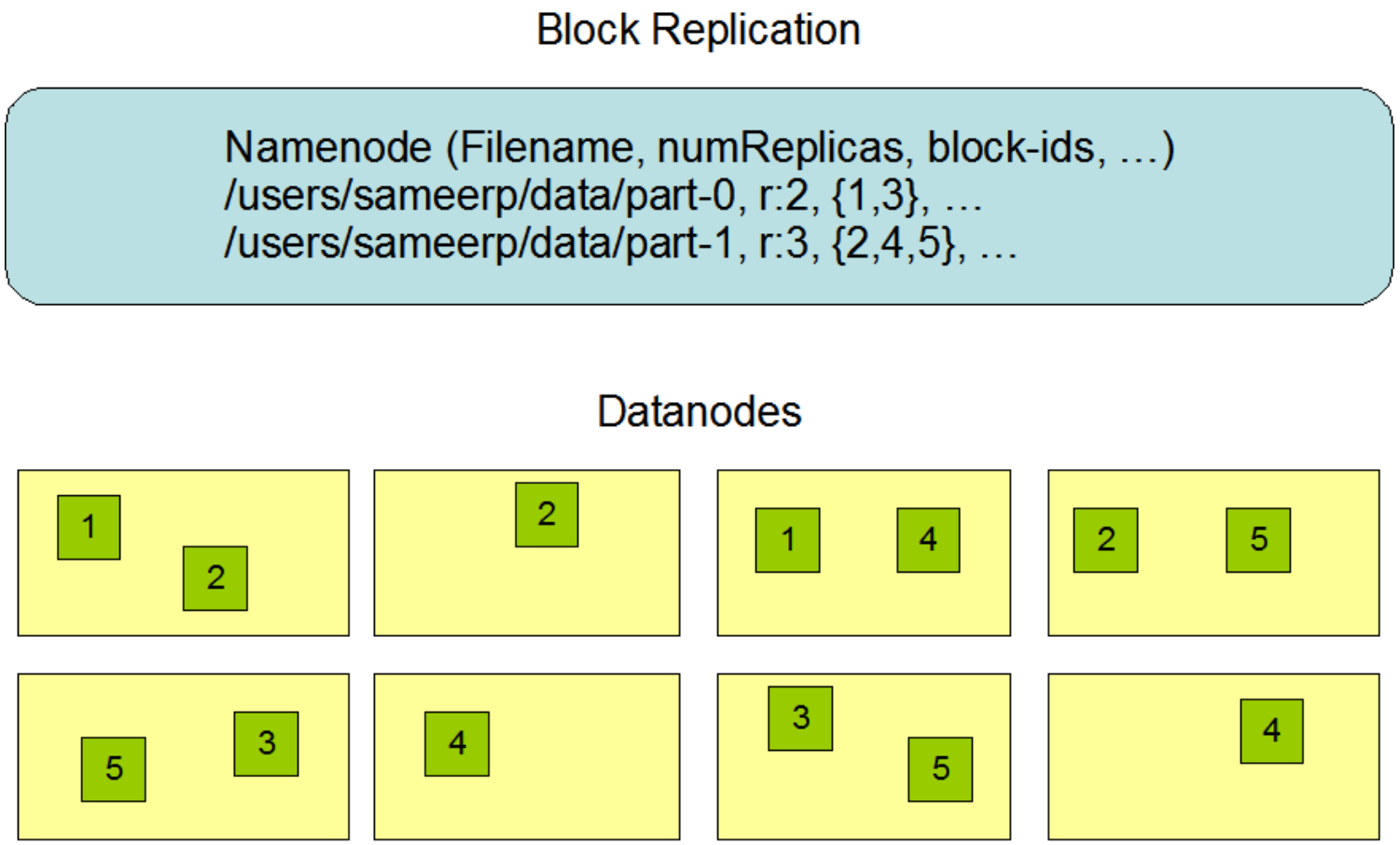
Datanode

- Много
- Хранит блоки данных
- Отвечает за чтение и запись
- Отправляет heartbeat для namenode

Бывает еще Secondary Namenode



Репликация

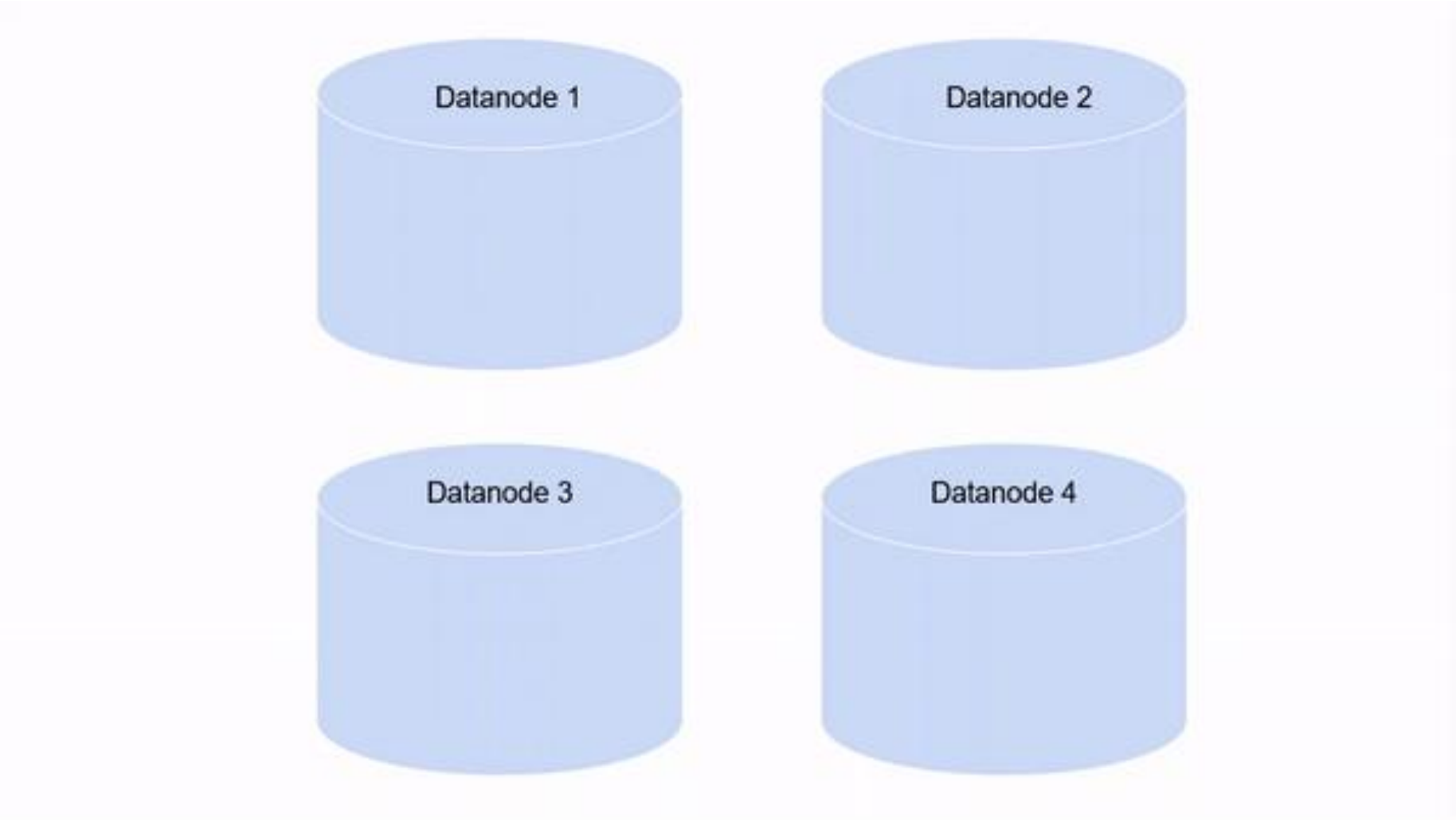


Обеспечивает

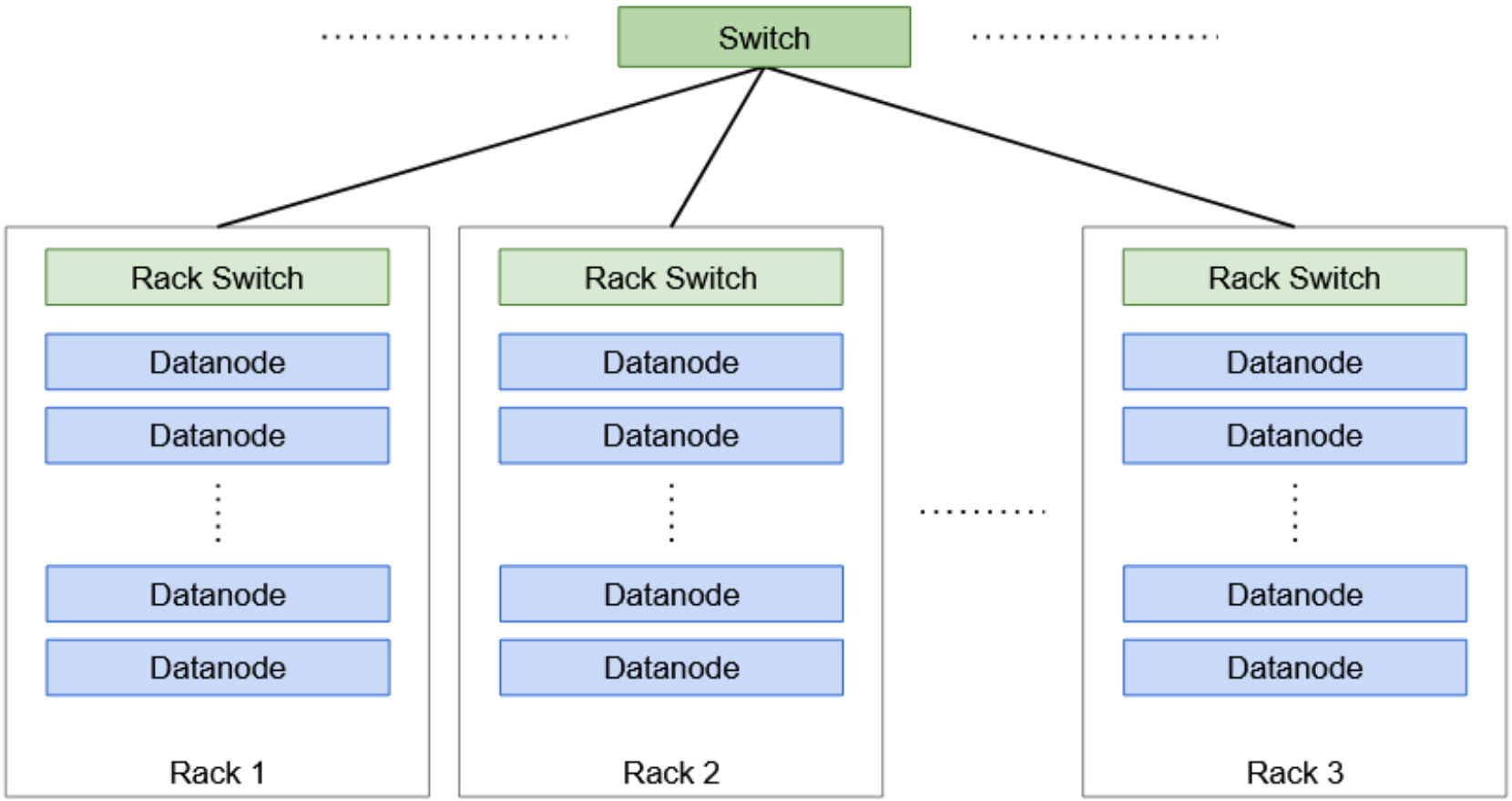
- **отказоустойчивость**
- **балансировку нагрузки**
- **параллельное чтение**

Что будет, если умрет датанода?

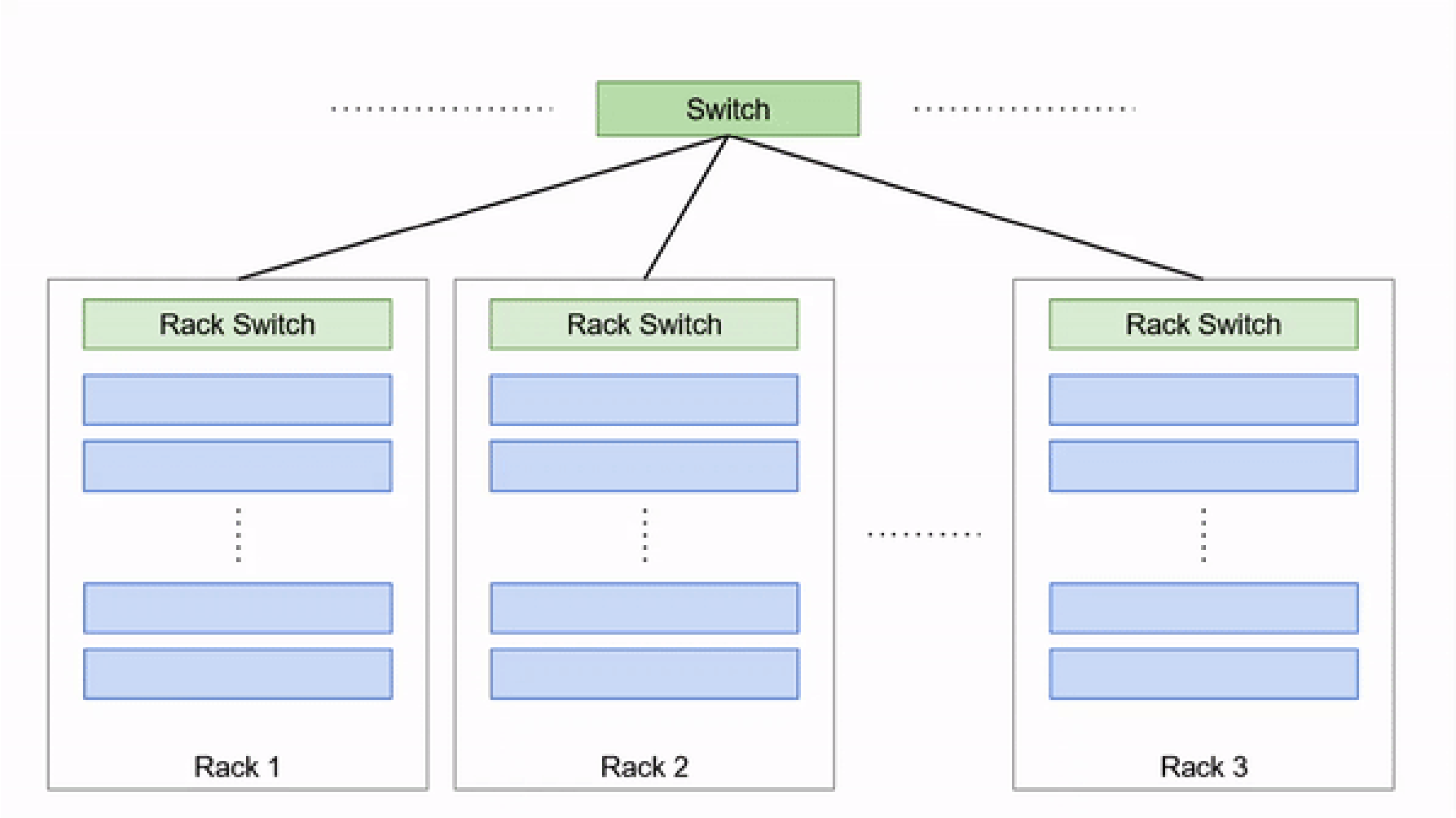
Как хранятся данные в HDFS



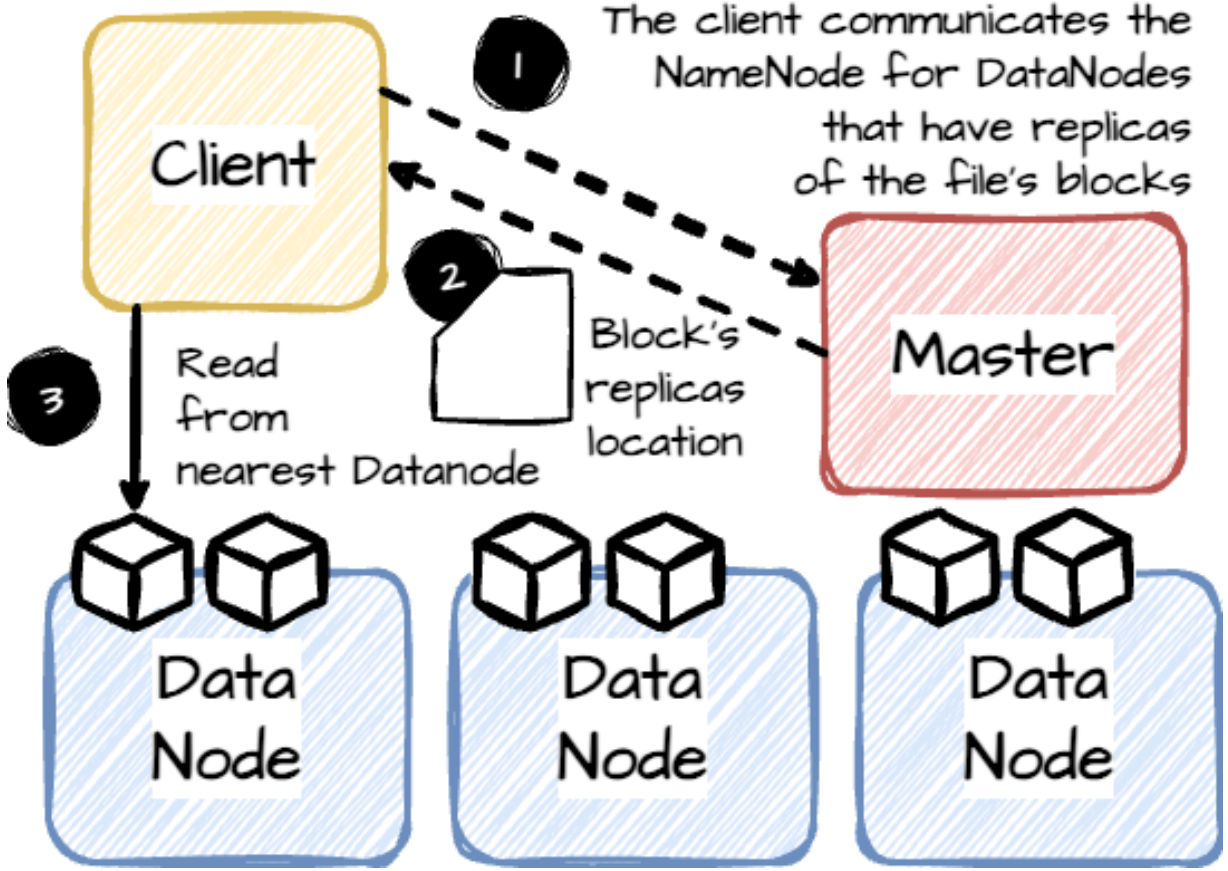
Rack awareness

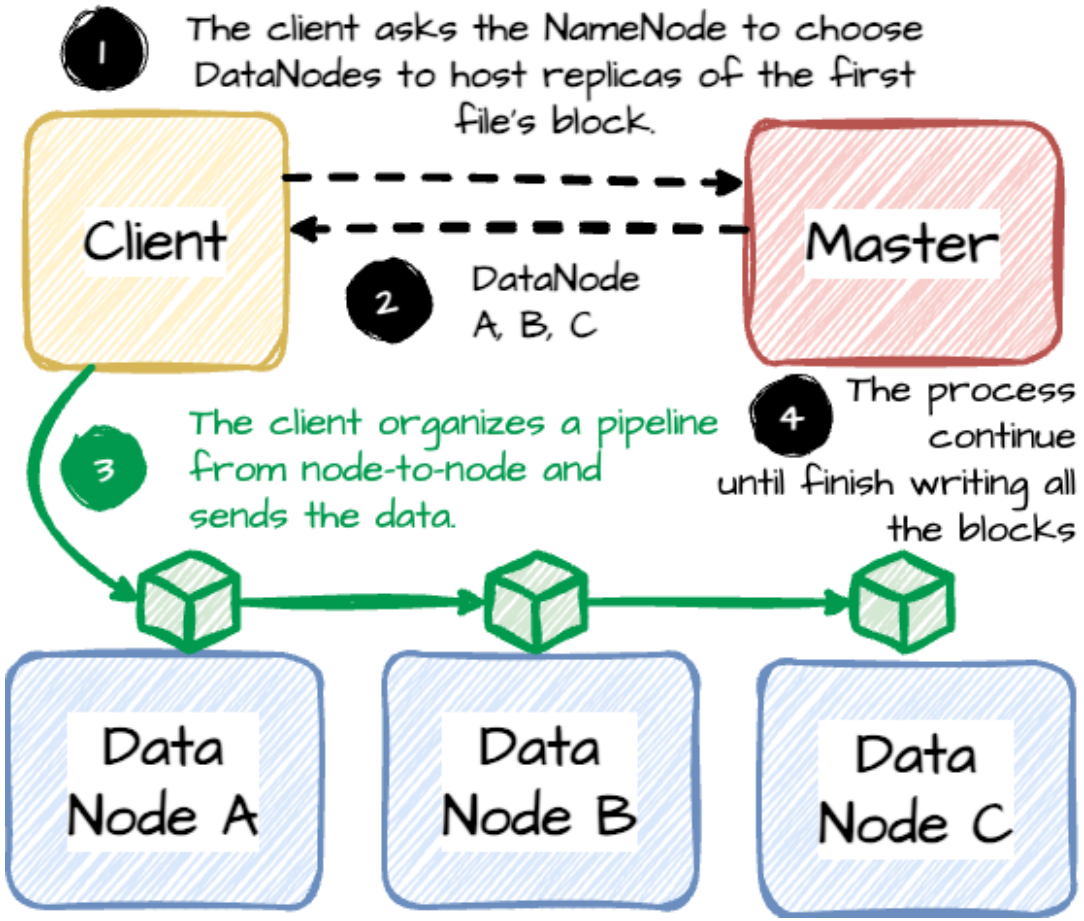


Rack awareness



Как происходит работа с данными



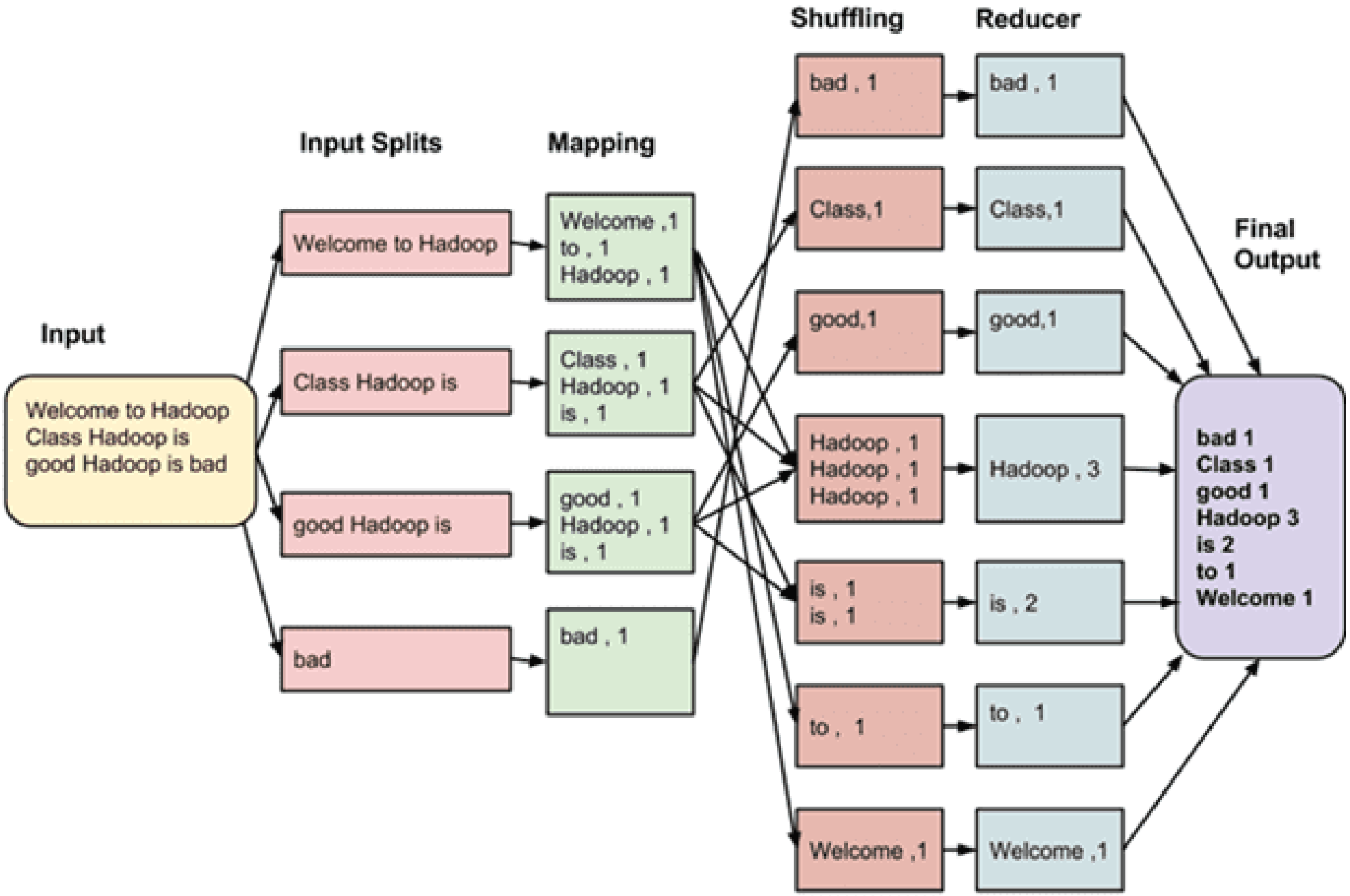


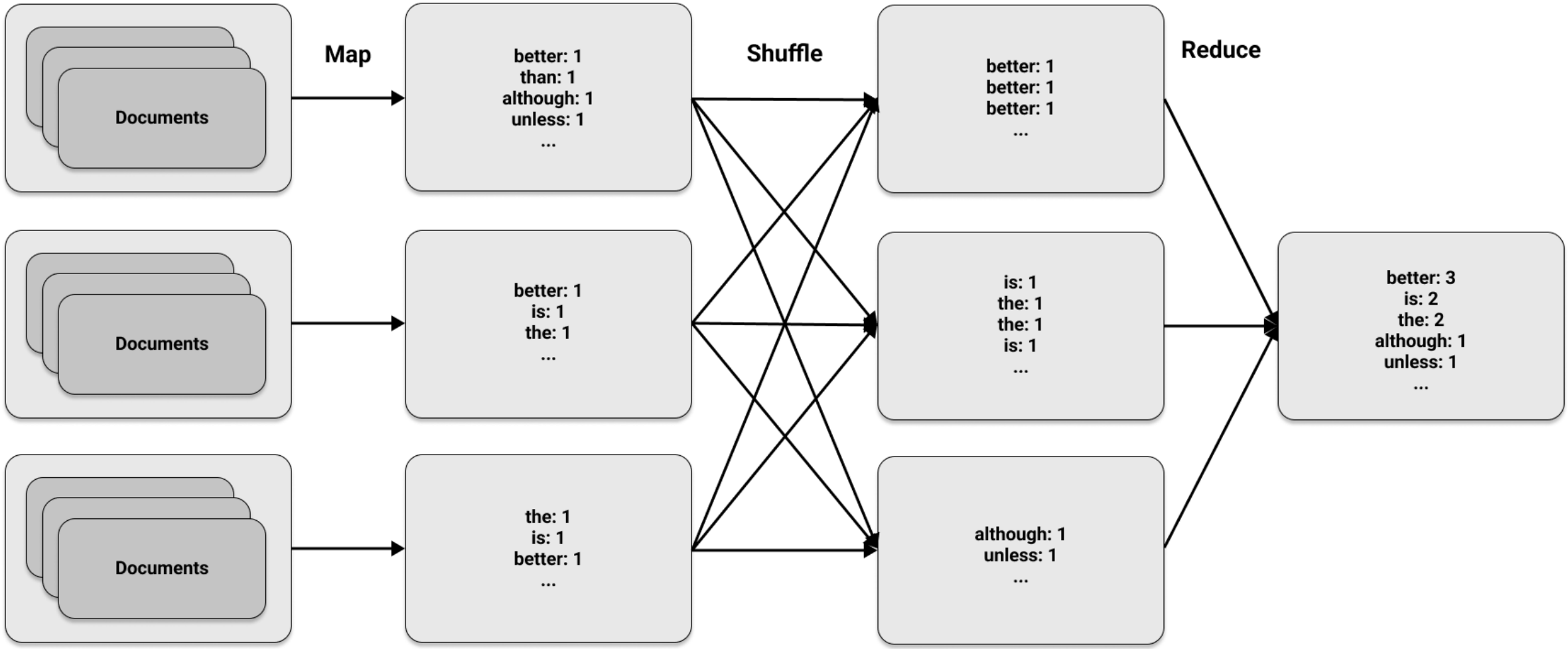
MapReduce

Apache MapReduce — модель обработки **больших данных**, в которой вычисления выполняются **параллельно** на множестве узлов.

Основные этапы работы MapReduce:

- **Map (Отображение):** Входные данные разбиваются, и мапперы преобразуют их в промежуточные пары «ключ-значение».
- **Shuffle (Сортировка/Группировка):** Система сортирует и группирует пары по ключу, распределяя их между редьюсерами.
- **Reduce (Сокращение):** Редьюсеры берут сгруппированные данные и агрегируют их, формируя конечный результат.


Input
Welcome to Hadoop
Class Hadoop is
good Hadoop is bad
Input Splits
Welcome to Hadoop
Class Hadoop is
good Hadoop is
bad
Mapping
Welcome ,1
to , 1
Hadoop , 1
Class , 1
Hadoop , 1
is , 1
good , 1
Hadoop , 1
is , 1
bad , 1
Shuffling
bad , 1
Class,1
good,1
Hadoop , 1
Hadoop , 1
Hadoop , 1
is , 1
is , 1
to , 1
Welcome ,1
Reducer
bad , 1
Class,1
good,1
Hadoop , 3
is , 2
to , 1
Welcome ,1
Final Output
bad 1
Class 1
good 1
Hadoop 3
is 2
to 1
Welcome 1

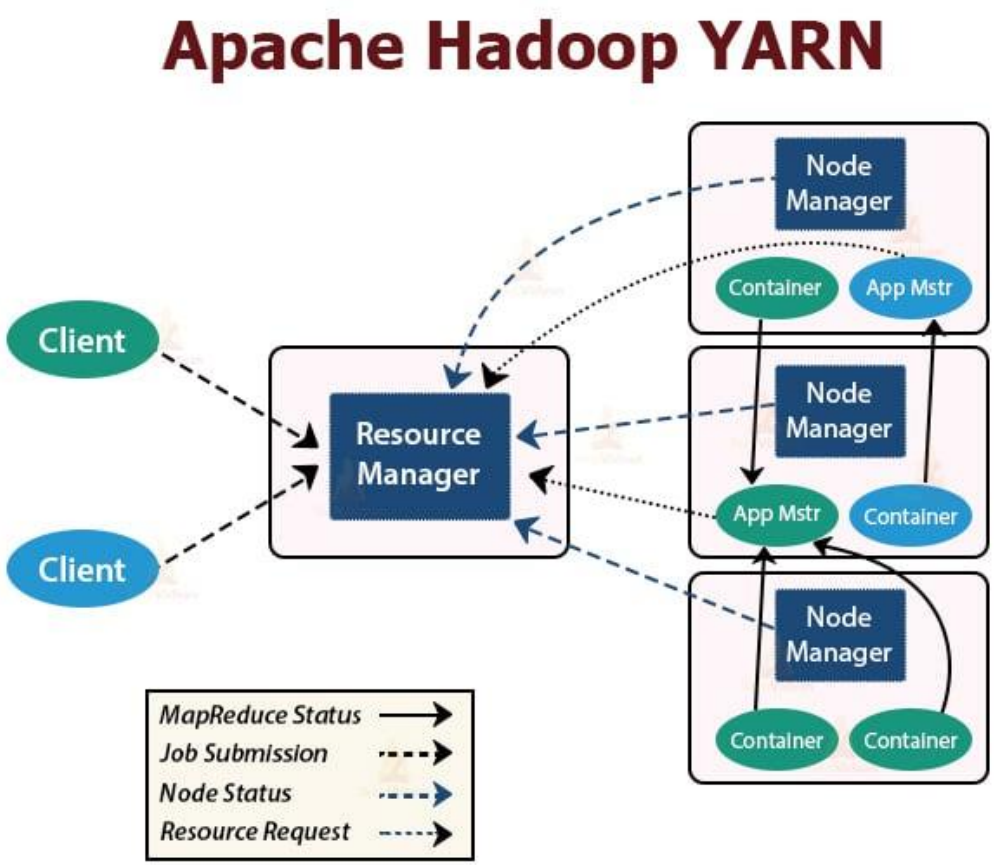

Documents
Documents
Documents
Map
better: 1
than: 1
although: 1
unless: 1
...
better: 1
is: 1
the: 1
...
the: 1
is: 1
better: 1
...
Shuffle
better: 1
better: 1
better: 1
...
is: 1
the: 1
the: 1
is: 1
...
although: 1
unless: 1
...
Reduce
better: 3
is: 2
the: 2
although: 1
unless: 1
...

YARN

YARN – менеджер ресурсов кластера

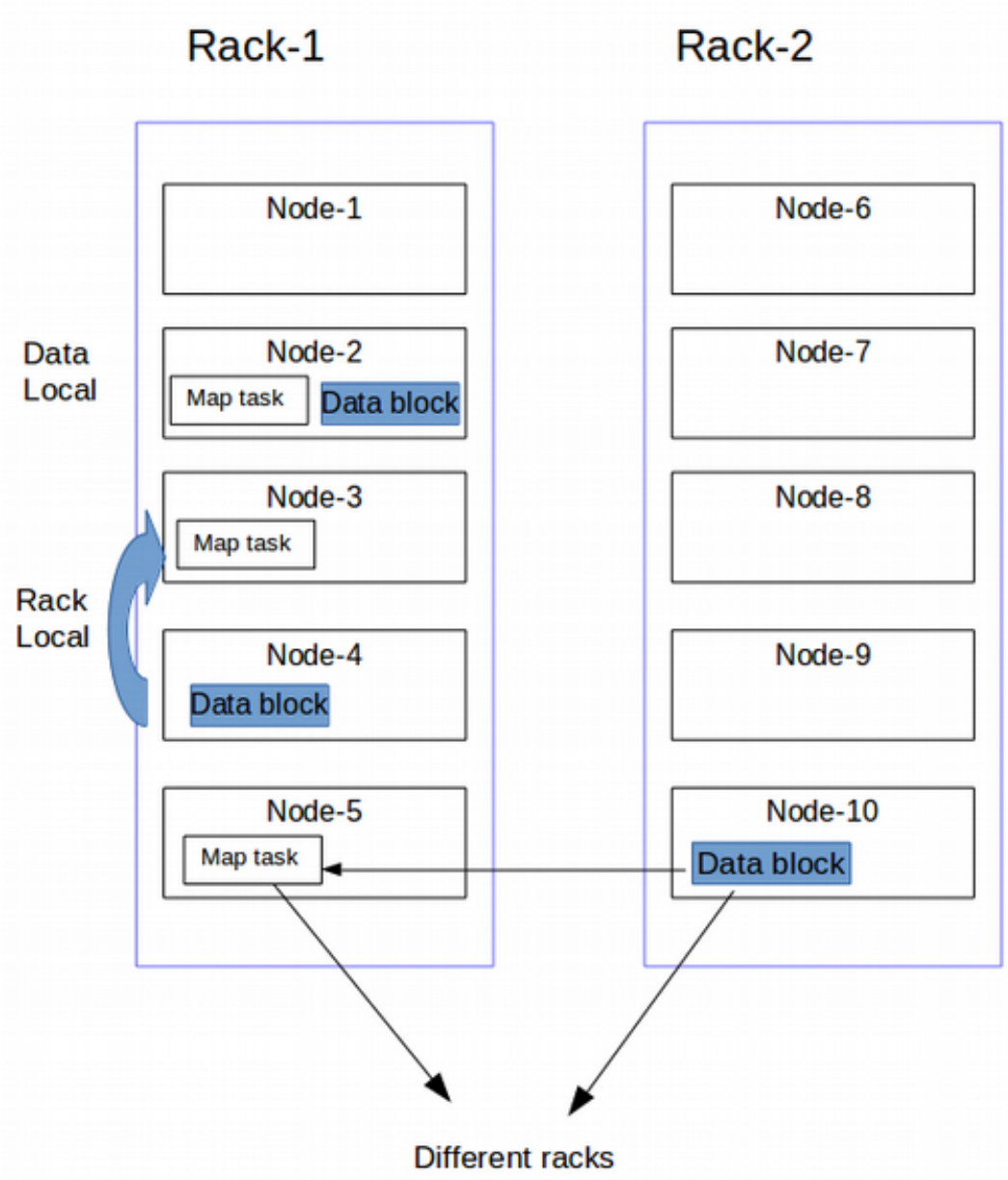
Задачи:

- Распределение CPU и памяти
- Запуск задач
- Управление очередями
- Мониторинг выполнения



Почему HDFS – быстро?

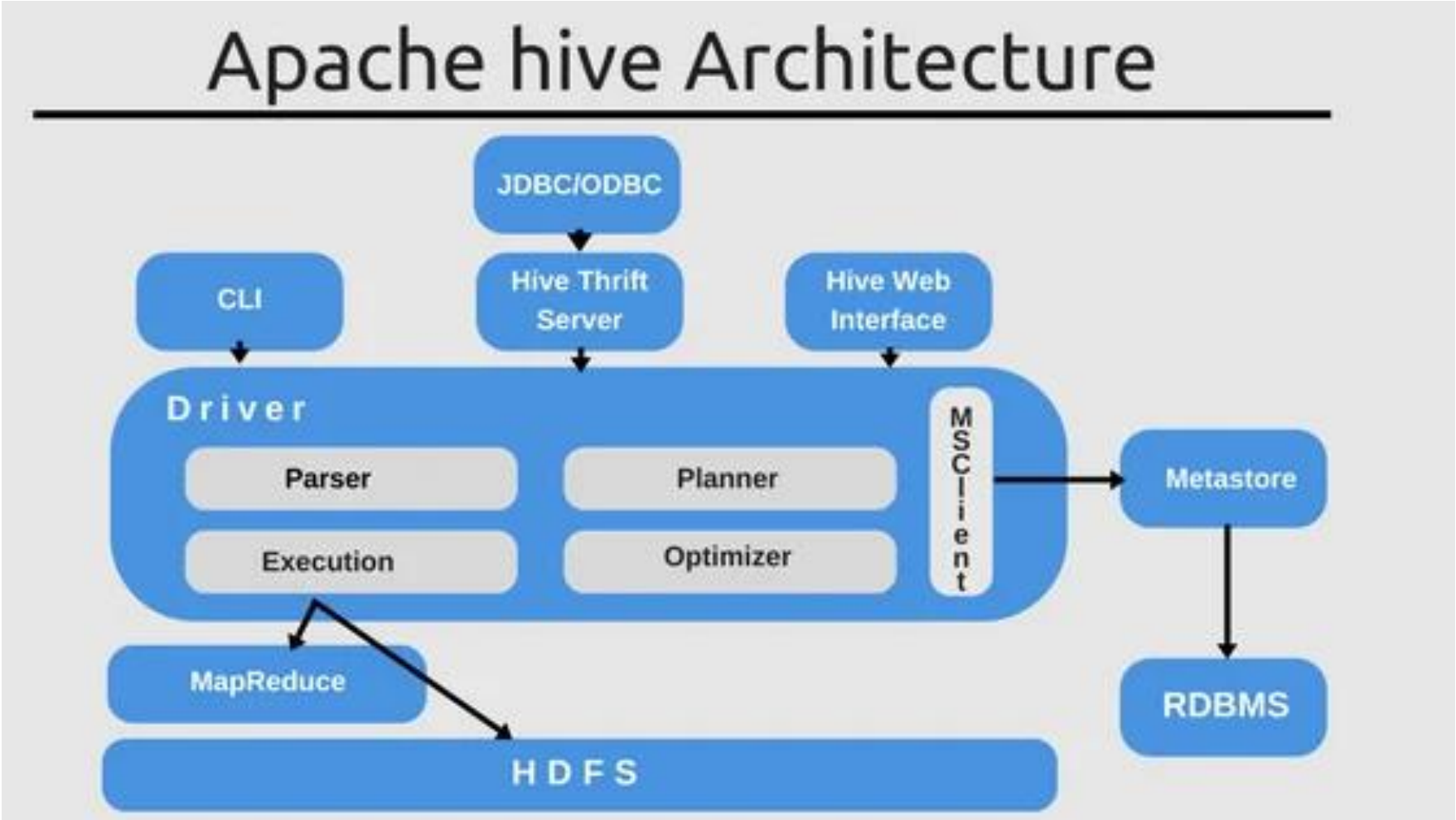
Локальность данных (Data Locality) в HDFS — это фундаментальный принцип работы Hadoop, который заключается в выполнении вычислений на тех же узлах, где физически хранятся данные.



HDFS CLI

Смотрим команды

Apache Hive

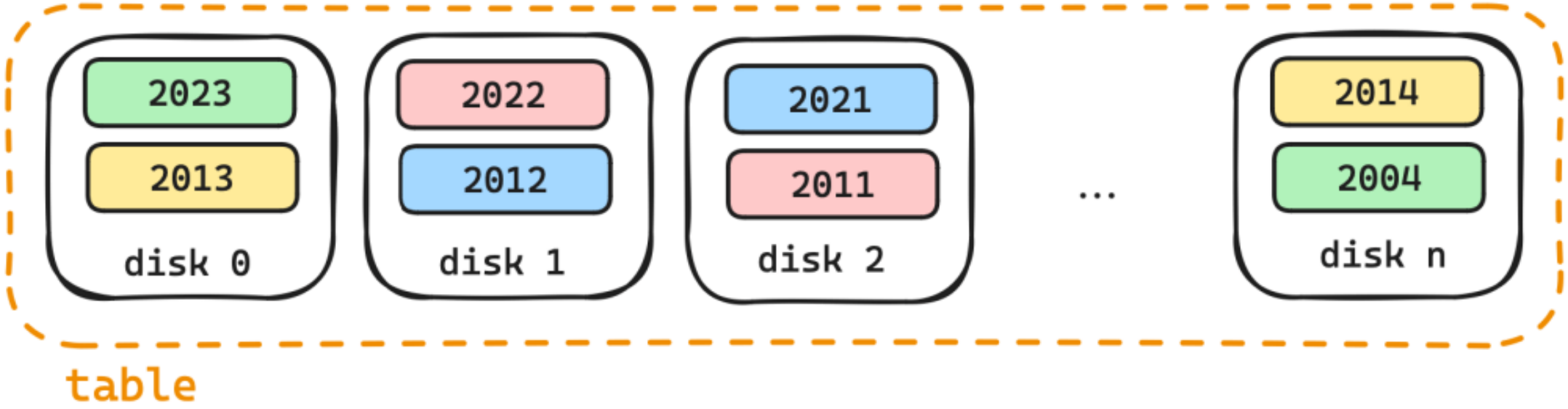

Apache hive Architecture
JDBC/ODBC
CLI
Hive Thrift Server
Hive Web Interface
Driver
Parser
Planner
Execution
Optimizer
MSClient
Metastore
MapReduce
RDBMS
HDFS

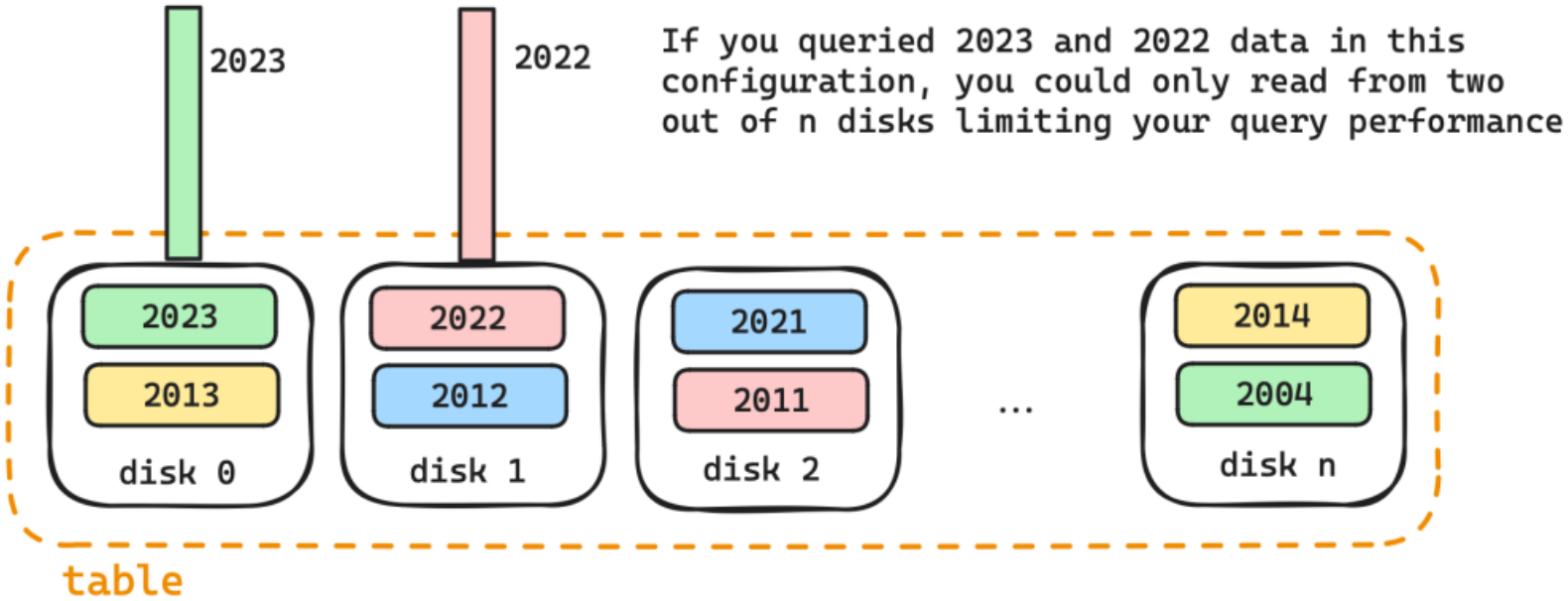
Apache Hive Syntax

```
SELECT user.*
FROM user
WHERE user.active = 1;
```

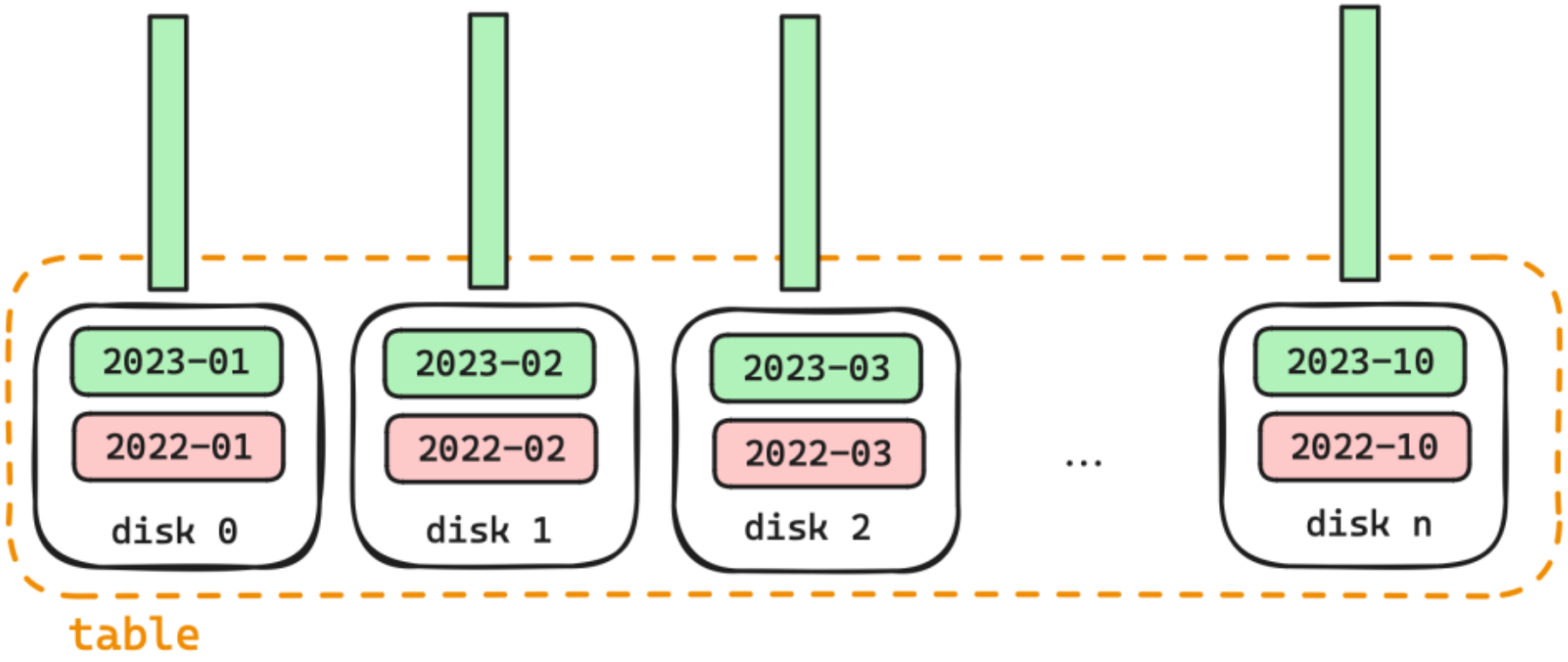
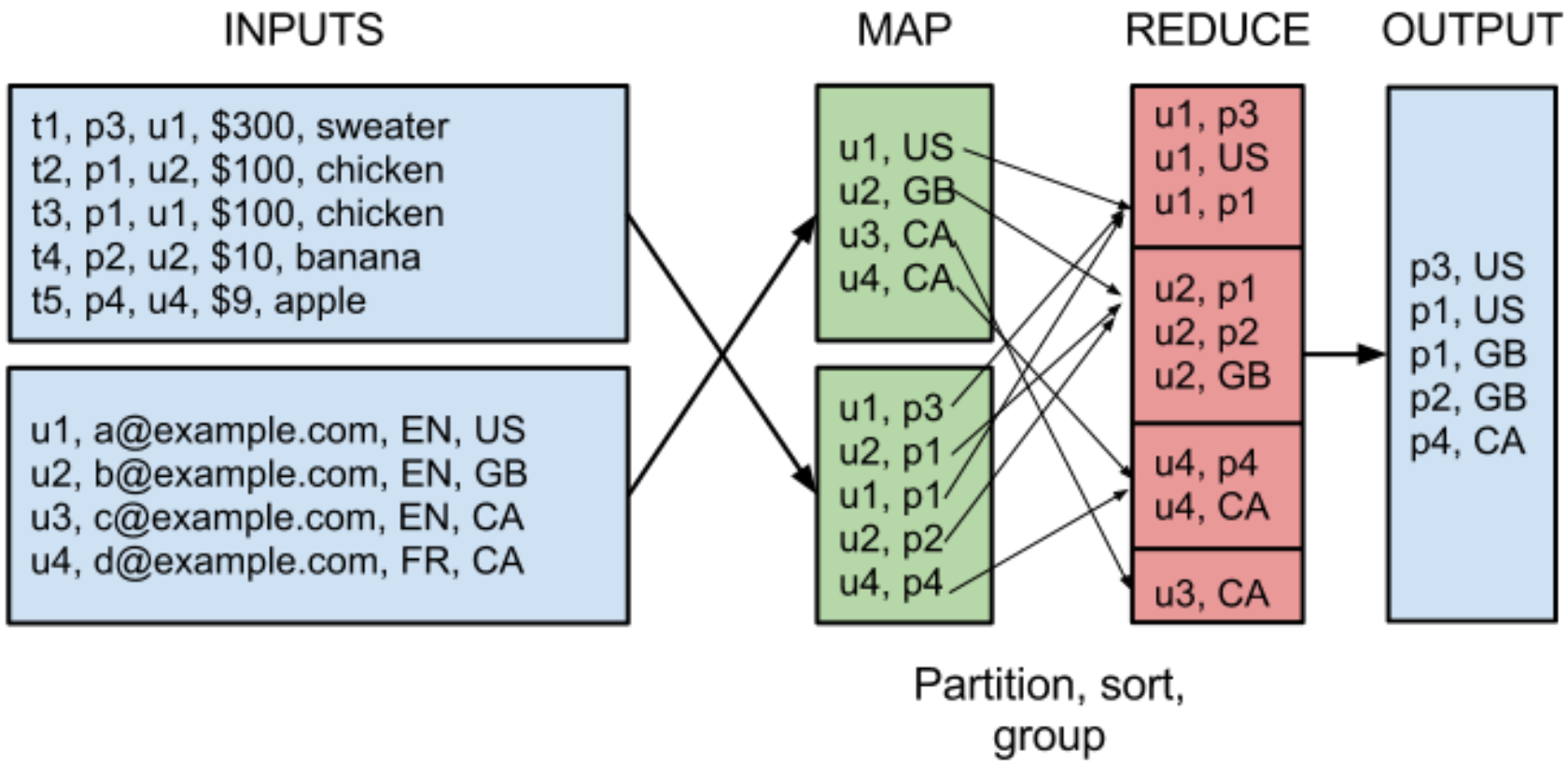
```
INSERT OVERWRITE TABLE pv_gender_agg
SELECT pv_users.gender, count(DISTINCT pv_users.userid), count(DISTINCT pv_users.ip)
FROM pv_users
GROUP BY pv_users.gender;
```

```
INSERT OVERWRITE TABLE xyz_com_page_views
SELECT page_views.*
FROM page_views
WHERE page_views.date >= '2008-03-01' AND page_views.date <= '2008-03-31' AND
      page_views.referrer_url like '%xyz.com';
```





If you queried 2023 and 2022 data in this configuration, you could only read from two out of n disks limiting your query performance


2023-01
2022-01
disk 0
2023-02
2022-02
disk 1
2023-03
2022-03
disk 2
...
2023-10
2022-10
disk n
table

Map Reduce Join